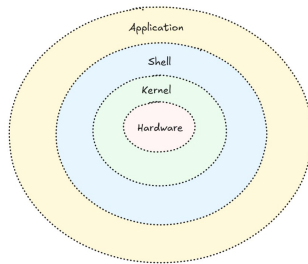


SHELL SCRIPTING



--> Shell is a command line interface (CLI) which will act as a medium to interact with operating system through text or commands

Types of Shell

1. GUI Shell

--> This kind of shell provides a graphical interface for the user, which they will be using to interact with an Operating system through menus or icons

2. CLI Shell

--> This kind of shell provides the interface for the user to interact with the operating system in the form of text or commands

Different types of CLI Shell

1. sh (Bourne Shell)
2. bash (Bourne again shell)
3. csh (cshell)
4. ksh (Korn Shell)

1. sh (Bourne Shell)

It is the original unix shell, It provides basic scripting features

2. bash (Bourne again shell)

It is the latest version of the Bourne Shell

3. csh (cshell)

It is a unix shell with syntax resembling to "C programming language"

4. ksh (Korn Shell)

It was developed by David Korn, It combines features of both sh (bourne shell) and csh (cshell)

Shell Script

--> Shell script is an executable file
--> It contains multiple shell commands which will get executed sequentially
--> We are using the shell scripting to automate the daily task
--> Extension for script file: .sh

Commands in shell Scripting

--> To list the available shells on the system: `cat /etc/shells`

--> To know which shell the system is using : `echo $0`

--> To check the location if the bash : `which bash`

Creation of Shell Script File

--> Step1: Create a file with ".sh" extension

--> Step2: Check the permission of the file, Grant the executable permission for the created script file

```
chmod u+x file_name
```

--> Step3: Write the shell script inside a file using the commands(nano/vi/vim)

*** Whenever we start writing the scripts inside the shell scripting file, the first line of the file will be starting with a "Shebang Operator" [#!] followed by the location where the bash is present

--> Step4: Execute/Run the shell scripting file

Way1: By providing the absolute path, i.e the exact location of the file from the top most directory

```
/home/<defaultuser-dir>/Desktop/script1.sh
```

Way2: Present in the current location where file is available, you can run the command

```
./script1.sh
```

Script1.sh

```
chmod u+x script1.sh
```

```
#!/bin/bash  
  
echo "Good Evening"  
echo "How was your day"
```

Comments

--> Line of code/instructions which are present in the file but it will not get executed
--> It increases the readability and understandability of the files as it acts as an description for the code/script

- * To comment a single line use a symbol that is #
- * To comment multiple lines

```
:<<'COMMENT'  
.....  
.....  
COMMENT
```

Script2.sh (To comment a single line)`

```
#!/bin/bash  
  
# print the following statements  
echo "Hi everyone"  
echo "How are you"
```

Script3.sh (To comment a multiple line)`

```
#!/bin/bash  
  
:<<'COMMENT'  
echo "Hi everyone"  
echo "How are you"  
COMMENT
```

VARIABLES

Variables are the named memory location which can store data or values and it can be changes "n" number of times during execution
--> It is used to store the data and values

There are different types of variables

1. Environment variable
2. System variable
3. Userdefined variable

"\$" is used to utilize the data stored in a variable

- * Environment variable
Environment variable are the special variable that holds the information about the system environment

Script4:

```
#!/bin/bash  
  
echo $HOME  
echo $USER  
echo $PWD  
echo $TERM  
echo $SHELL
```

- * System Variable

System variable is a subsystem of the Environment variables that provide the information about the system configurations

Script5:

```
#!/bin/bash  
  
echo $MACHTYPE  
echo $UID  
echo $HOSTNAME  
echo $LOGNAME
```

- * Userdefined variable

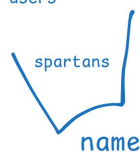
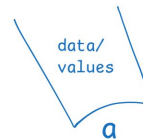
These are the variables defined/created by the users

Script6:

```
#!/bin/bash  
  
name="Spartans"  
age=20  
echo "The name is: $name"  
echo "The age is: $age"
```

Dynamic Input

```
#!/bin/bash  
  
echo "Enter the name"  
read name  
echo "Enter the age"  
read age  
echo "The name is: $name"  
echo "The age is: $age"
```



String Values

```
#!/bin/bash

echo "Enter the name"
read name
echo "the name is $name"
echo 'the name is $name'
```

O/P

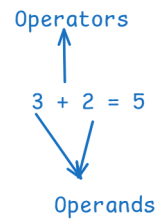
Spartans
\$name

Operators

These are the symbols which are used to perform a specific task

Different types of Operators

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. String Operators
5. File Operators



* Arithmetic Operators

Used to perform Mathematical Operations

--> Addition	+
--> Subtraction	-
--> Multiplication	*
--> Division (quotient)	/
--> Modulus	%

Syntax:

```
$((expression))
```

```
#!/bin/bash

echo "Enter the first value"
read num1
echo "Enter the second value"
read num2

# to perform addition
sum=$((num1+num2))

# to perform subtraction
diff=$((num1-num2))

# to perform multiplication
mul=$((num1*num2))

# to perform division
# to get quotient
quot=$((num1/num2))

# to get remainder
mod=$((num1%num2))

# To display the results

echo "Addition: $sum"
echo "Subtraction: $diff"
echo "Multiplication: $mul"
echo "Quotient: $quot"
echo "Modulus: $mod"
```

* Relational Operators

--> It is used to compare two different values

--> It will return the boolean values that is "true" (1) or false (0)

1. equals	-eq	==
2. not equals	-ne	!=
3. greater than	-gt	>
4. lesser than	-lt	<
5. greater than or equals	-ge	>=
6. lesser than or equals	-le	<=

```
#!/bin/bash
```

```
echo "Enter the first value"
read a
echo "Enter the second value"
read b
```

```
# equals
echo "a==b: $((a==b))
```

```
# not equals
echo "a!=b: $((a!=b))
```

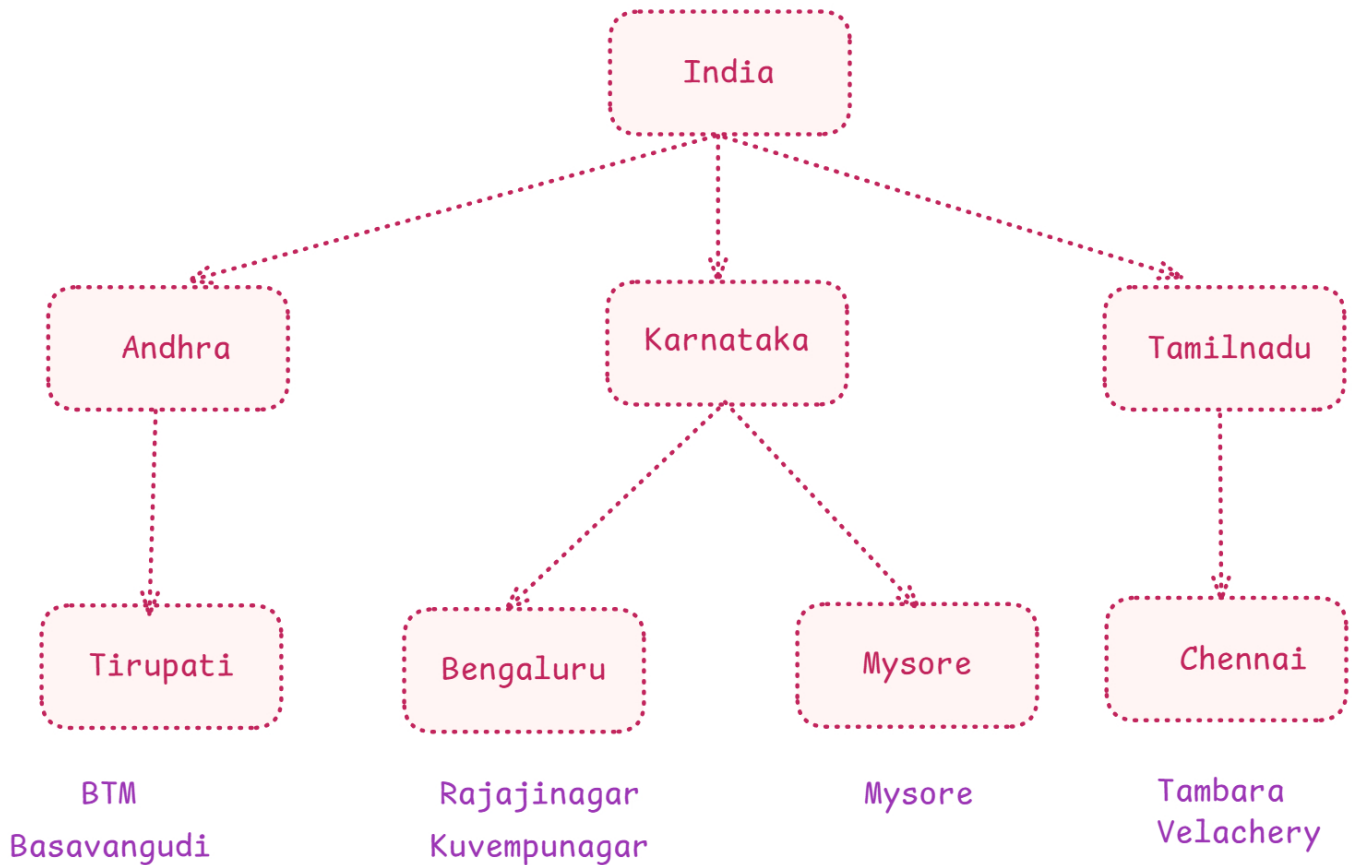
```
# lesser than
echo "a<b: $((a<b))
```

```
# greater than
echo "a>b: $((a>b))
```

```
# lesser than or equals
echo "a<=b: $((a<=b))
```

```
# greater than or equals
echo "a>=b: $((a>=b))
```

TASK



1. Create above directory Hierarchy
2. Move "BTM" and "Basavangudi" from "Tirupati" to "Bengaluru"
3. Copy "Basavangudi" from "Bengaluru" to "Mysore"
4. Rename "Tambara" in "Chennai" to "Tambaram"

Note: Create "Directories" and "Sub-directories" in a Single Command

```
#!/bin/bash

mkdir -p India/Andhra/Tirupati India/Karnataka/Bengaluru India/Karnataka/Mysore India/Tamilnadu/Chennai

# Create files inside the directories

touch India/Andhra/Tirupati/BTM
touch India/Andhra/Tirupati/Basavangudi
touch India/Karnataka/Bengaluru/Rajajinagar
touch India/Karnataka/Bengaluru/Kuvempunagar
touch India/Karnataka/Mysore/Mysore
touch India/Tamilnadu/Chennai/Tambara
touch India/Tamilnadu/Chennai/Velachery

# Move BTM and Basavangudi from Tirupati to Bengaluru

mv India/Andhra/Tirupati/BTM India/Karnataka/Bengaluru
mv India/Andhra/Tirupati/Basavangudi India/Karnataka/Bengaluru

# Copying Basavangudi from Bengaluru to mysore

cp India/Karnataka/Bengaluru/Basavangudi India/Karnataka/Mysore/

# Rename Tambara to Tambaram

mv India/Tamilnadu/Chennai/Tambara India/Tamilnadu/Chennai/Tambaram
```

* Logical Operators

Logical operators are used to combine multiple conditions

Types of Logical Operators

1. Logical AND ("&" or "&&")
2. Logical OR ("-o" or "||")
3. Logical NOT ("!")

Logical Table

AND

1

I/P		O/P
A	B	
0	0	0
0	1	0
1	0	0
1	1	1

- > If both the conditions are true, then the output is true (1)
- > If either one of the condition is false (0), then the output will be false

OR

I/P		O/P
A	B	
0	0	0
0	1	1
1	0	1
1	1	1

- > If either one of the condition is true, then the output is true
- > If both the conditions are false, the output is false

NOT

I/P	O/P
A	NOT A
0	1
1	0

```
#!/bin/bash
```

```
echo "Enter the first number"
read a
echo "Enter the second number"
read b
```

```
# Logical AND
echo "a AND b: $((a && b))"
```

```
# Logical OR
echo "a OR b: $((a || b))"
```

```
# Logical NOT
echo "NOT a: $((!a))"
echo "NOT b: $((!b))"
```

* String Operators

These are used to compare two strings

1. Equals =
2. Not Equal !=
3. Null -z
4. Not Null -n

* File Operators

File operators are used to check the information files or directories before working with shell scripting

1. -e --> file exist
2. -r --> file is readable
3. -w --> file is writable
4. -x --> file is executable
5. -d --> Directory exists
6. -s --> file is not empty

Conditional Statements

A conditional statement is a decision making tool in programming or shell scripting. It lets your code to choose what to do based on a condition

1. if statement

This statement is used to execute a block of code only if the condition is true, if the condition is false the code inside if-statement is not going to get executed

Syntax:

```
if [ condition ];  
then  
    statement  
fi
```

2. if-else statement

The if-else statement is used when you want to execute a block of code if the condition is true, if the condition is false different block of code will be executed

Syntax:

```
if [ condition ];  
then  
    statement1;  
else  
    statement2;  
fi
```

3. elif statements

The elif statement is used to execute multiple conditions sequentially
--> It allows you to handle more than two possible outcomes by checking one after the other

Syntax:

```
if [ condition ];  
then  
    statement1  
  
elif [ condition2 ];  
then  
    statement2  
else  
    statement3  
fi
```

4. nested-if statement

A nested-if is a statement placed inside another if-statement

Syntax:

```
if [ condition ];  
then  
    if [ condition ];  
    then  
        statement1  
    else  
        statement2  
    fi  
else  
    statement3  
fi
```

Execution of Scripts

Script1:

```
#!/bin/bash  
  
echo "Enter the number"  
read num  
if [ $num -gt 100 ]; then  
    echo "The number is greater than 100"  
fi
```

Script2: If temperature is greater than 85 it should get executed, if it is below 85 it should not print a statement

```
#!/bin/bash  
  
echo "Enter the temperature"  
read temp  
if [ $temp -gt 85 ]; then  
    echo "It is really hot"  
fi
```

Script3: Checking a number is even or odd

```
#!/bin/bash  
  
echo "Enter a number"  
read num  
if [ $((num%2)) -eq 0 ]; then  
    echo "The number $num is even"  
fi
```

* if-else statement

Script4: Checking a number is even or odd

```
#!/bin/bash  
  
echo "Enter a number"  
read num  
if [ $((num%2)) -eq 0 ]; then  
    echo "The number $num is even"  
else  
    echo "This number $num is odd"  
fi
```

Script5: If the age is greater than 18 "Print as adult"
else "Print you are a kid"

```
#!/bin/bash  
  
echo "Enter the age"  
read age  
if [ $age -ge 18 ]; then  
    echo "You are an adult"  
else  
    echo "You are a kid"  
fi
```

* elif staments

Script6: Check whether the number is positive, negative or zero

```
#!/bin/bash

echo "Enter a number"
read num
if [ $num -gt 0 ]; then
echo "The number $num is positive"
elif [ $num -lt 0 ]; then
echo "The number $num is negative"
else
echo "The number $num is Zero"
fi
```

Script7: Check if the temperature is below 32, print "Its freezing outside"
--> If the temperature is below 32 and 60, print "Its bit chilly"
--> If the temperature is above 60, Print "Its warm outside"

```
#!/bin/bash

echo "Enter the temperature"
read temp
if [ $temp -lt 32 ]; then
echo "Its freezing outside"
elif [ $
```

```
Script8: If the number is between 10 and 20 or greater than 50, Print the number is
        valid
        --> If not, "Print the number is not valid"
```

```
#!/bin/bash

echo "Enter a number"
read num

if { [ $num -ge 10 ] && [ $num -le 20 ]; } || [ $num -gt 50 ]; then
echo "the number is valid"
else
echo "the number is invalid"
fi
```

OR

```
#!/bin/bash

echo "Enter a number"
read num

if [ \( $num -ge 10 -a $num -le 20 \) -o $num -gt 50 ]; then
echo "the number is valid"
else
echo "the number is invalid"
fi
```

* nested-if statement

Script9: Comparison of three numbers, to identify which is greater

```
#!/bin/bash

echo "Enter the first number"
read num1
echo "Enter the second number"
read num2
echo "Enter the third number"
read num3

if [ $num1 -gt $num2 ]; then

    if [ $num1 -gt $num3 ]; then
        echo "$num1 is the greatest of all the three numbers"
    else
        echo"$num3 is greatest of all the three numbers"
    fi

else

    if [ $num2 -gt $num3 ]; then
        echo "$num2 is the greatest of all the three numbers"
    else
        echo "$num3 is the greatest of all the three numbers"
    fi
fi
```

Script10:
Check if a person is eligible for a loan based on their monthly income
* Criteria
--> Age must be between 18 and 60 years
--> Income must be at-least 30k per month

```
#!/bin/bash

echo "Enter your age"
read age
echo "Enter your monthly income"
read income

if [ $age -ge 18 ] && [ $age -le 60 ]; then

    if [ $income -ge 30000 ]; then
        echo "Loan is approved"
    else
        echo "Loan is denied due to insufficient income"
    fi

else
echo "Loan is denied due to age restrictions"
fi
```

* String Operator

These are used to compare two strings

1. Equals =
2. Not equals !=
3. Null -z
4. Not null -n

Script11: Equals

```
#!/bin/bash

echo "Enter the string1"
read str1
echo "Enter the string2"
read str2

if [ "$str1" = "$str2" ]
echo "Both strings are equal"
fi
```

Script12: Not-Equals

```
#!/bin/bash

echo "Enter the string1"
read str1
echo "Enter the string2"
read str2

if [ "$str1" = "$str2" ]
echo "Both strings are not equal"
fi
```

Script13: null

```
#!/bin/bash

echo "Enter the string value"
read str1
if [ -z "$str1" ]; then
echo "String is empty"
fi
```

Script14: not-null

```
#!/bin/bash

echo "Enter the string value"
read str1
if [ -n "$str1" ]; then
echo "String is not empty"
fi
```

* String Operator

These are used to compare two strings

1. Equals =
2. Not equals !=
3. Null -z
4. Not null -n

Script11: Equals

```
#!/bin/bash

echo "Enter the string1"
read str1
echo "Enter the string2"
read str2

if [ "$str1" = "$str2" ]
echo "Both strings are equal"
fi
```

Script12: Not-Equals

```
#!/bin/bash

echo "Enter the string1"
read str1
echo "Enter the string2"
read str2

if [ "$str1" = "$str2" ]
echo "Both strings are not equal"
fi
```

Script13: null

```
#!/bin/bash

echo "Enter the string value"
read str1
if [ -z "$str1" ]; then
echo "String is empty"
fi
```

Script14: not-null

```
#!/bin/bash

echo "Enter the string value"
read str1
if [ -n "$str1" ]; then
echo "String is not empty"
fi
```

* File Operators

Script15: file exists

```
#!/bin/bash

echo "Enter the file path"
read file_path

if [ -e "$file_path" ]; then
echo "File exists"
else
echo "File is not present"
fi
```

Script16: file readable

```
#!/bin/bash

echo "Enter the file path"
read file_path

if [ -r "$file_path" ]; then
echo "File is readable"
else
echo "File is not readable"
fi
```

Script17: file writable

```
#!/bin/bash

echo "Enter the file path"
read file_path

if [ -w "$file_path" ]; then
echo "File is writable"
else
echo "File is not writable"
fi
```

Script18: file executable

```
#!/bin/bash

echo "Enter the file path"
read file_path

if [ -x "$file_path" ]; then
echo "File is executable"
else
echo "File is not executable"
fi
```

Script19: directory exists

```
#!/bin/bash

echo "Enter the file path"
read dir_path

if [ -d "$dir_path" ]; then
echo "Directory is present"
else
echo "Directory is not present"
fi
```

Script20: file is not empty

```
#!/bin/bash

echo "Enter the file path"
read file_path

if [ -s "$file_path" ]; then
echo "File is empty"
else
echo "File is not empty"
fi
```

Looping Statements

Looping is fundamental concept in programming allowing us to execute a block of code repeatedly based on condition

Types of Looping Statements

1. for loop
2. while loop
3. until loop

1. for loop

- > The for loop iterates over a list of items, executing a block of code for each time
- > When we know the starting and ending range then you are going to use this for loop

Syntax:

```
for var in list
do
# commands/statements
done
```

OR

Syntax:

```
for ((i=1; i<=n; i++))
do
#commands/statements
done
```

Script21:

```
#!/bin/bash

for i in 1 2 3 4 5
do
echo "$i"
done
```

Script22:

```
#!/bin/bash

for ((i=1; i<5; i++))
do
echo "$i"
done
```

* while loop (to print number from 1 to 5)

- > The while loop executes a block of code till the condition is true
- > Once the conditions becomes false it will come out of the loop
- > We are going to use the while loop when we don't know the starting and ending range

Syntax:

```
while [ condition ]
do
# commands/statements
done
```

Script23:

```
#!/bin/bash

i=1
while [ $i -le 5 ]
do
echo $i
i=$((i+1))
done
```

Script24: If the entered number is 0 it should come out of the loop
--> If it is not zero it should iterate in the loop itself

```
#!/bin/bash

number=1 # To get inside the loop for the first time
while [ $number -ne 0 ]
do
read -p "Enter the number here: " number
echo "You entered: $number"
done
echo "You entered 0: Exiting from the while loop"
```

* until loop

- > It is similar to while loop, but it executes as long as the condition is false
- > If the condition becomes true, it will come out of the loop

Syntax:

```
until [ condition ]
do
# commands/statements
done
```

Script24:

```
#!/bin/bash

i=1
until [ $i -gt 5 ]
do
echo $i
i=$((i+1))
done
```

Functions

A function is a reusable block of code that performs a specific task and it can be called multiple times within a script

Defining a function

There are two different ways to define a function

1. `function_name() {
 commands
}`
2. `function function_name{
 commands
}`

Calling the function

--> To call the function use the name of the function

```
function-name
```

Script25:

```
#!/bin/bash
echo "Executing the functions"
# Defining the function
hello() {
    echo "Hello Students!!!"
}

function Devops {
    echo "Welcome to DevOps"
}

# Calling the function
hello
Devops
```

Functions variable scope --> Accessibility of the script

--> Variables are classified into two type

- 1) Local Variable
- 2) Global Variable

--> Variables declared in a script by default it is a global variable

--> Variables declared directly inside the function block, it is also referred as global variable

--> Variables declared with a keyword "local" then it is called as Local variables

Note:

--> We can access global variable through out the script

--> The local variables are restricted to specific function block

Script26:

```
#!/bin/bash

my_function() {
    global_var="I am a global variable"
    local local_var="I am a local variable"
    echo $global_var
    echo $local_var
}

# Calling the function

my_function

# Outside the function

echo "Accessing the variables outside the function"
echo $global_var
echo $local_var
```

--> Passing arguments to the function

--> Arguments passed to a function are accessed using "\$1", "\$2", "\$3" ..etc
--> It is just like script arguments

Script27:

```
#!/bin/bash

my_function() {
    echo first argument: $1 : .....» Positional Arguments
    echo second argument: $2
}

my_function 5 8
```

Note: `echo $0` --> To get the name of the script
`echo $#` --> To get the count of the arguments

Note: `echo $0` --> To get the name of the script
`echo $#` --> To get the count of the arguments

Commands

1. `&> /dev/null` (It redirects the output or an error)
2. `which tool` (To check the location of the tool)
3. `id user_name` (To get the complete details of the user)
4. `cut -d ':' -f1 <file_name>`
 -d: delimiter
 -f1: field 1

 * To cut multiple Fields

 `cut -d: -f1,2,3 <file_name>` (To cut multiple fields)
5. `id -u user-name` (To get only the user id)

* To check user exist or not

```
#!/bin/bash

read -p "Enter the username: " user
if id "$user" &> /dev/null; then
echo "user exists"
else
echo "user does not exists"
fi
```

* To check tool exist or not if the tool is not present install the Tool

```
#!/bin/bash
echo "Enter the tool name"
read tool

# check if the tool exists or not
if which $tool &> /dev/null;
then
echo "$tool already exists"
else
echo "$tool not found, Installing it"
sudo apt install "$tool" -y &> /dev/null
echo "$tool installed"
fi
```

* Script to count number of Lines in a file

```
#!/bin/bash

:<<'COMMENT'
read -p "Enter the path: " file
if [ -f "$file" ]; then
lines=$(wc -l < "$file")
echo "Number of Lines: $lines"
else
echo "File is not present"
fi
COMMENT

read -p "Enter file name/path:" file

if [ -f "$file" ]; then
lines=$(cat "$file" | wc -l)
echo "Number of Lines: $lines"
else
echo "File is not present"
fi
```

* Script to delete the user except the default one

```
#!/bin/bash

echo "Enter a default username"
read defUser
users=$(cut -d: -f1 /etc/passwd)

for user in $users; do
user_id=$(id -u "$user")

if [ "$user_id" -ge 1000 ] && [ "$user_id" -le 60000 ]; then
if [ "$user" != "$defUser" ]; then
echo "Deleting user: $user"
sudo userdel -r "$user"
echo "$user deleted"
fi
fi
done
```

* To delete a specific user except the default and system user

```
#!/bin/bash

echo "Enter the username to delete"
read username

# Check user exist
if id "$username" &> /dev/null; then
user_id=$(id -u "$username")

if [ "$user_id" -gt 1000 ]; then
echo "Deleting the user: $username"
sudo userdel -r "$username"
echo "User $username deleted"
else
echo "Refusing to delete the user because the UID is less than 1000"
fi
else
echo "User $username doesnot exists"
fi
```

* Script to Deploy an application using Shell Scripting

```
#!/bin/bash

install_nginx(){
if which nginx &> /dev/null;
then
echo "Nginx is installed, proceed with deployment"
else
echo "nginx was not found"
sudo apt install nginx -y &> /dev/null
fi
}

start_nginx(){
if pgrep nginx &> /dev/null; then
echo "Nginx is already running"
else
echo "Nginx is not in the active state"
sudo systemctl start nginx
echo "Nginx is active and it is running"
fi
}

install_nginx
start_nginx

echo "Provide the url for the project"
read url
wget -O website.zip $url &> /dev/null
unzip website.zip &> /dev/null
echo "Extracted successfully"
ls

echo "Provide the path where the project is present in your system"
read path
echo "Deploying the project to nginx"
sudo cp -r $path/* /var/www/html
echo "Project installed in nginx"
echo "To see the website: Copy the public IP of the instance and paste it in the browser"
```